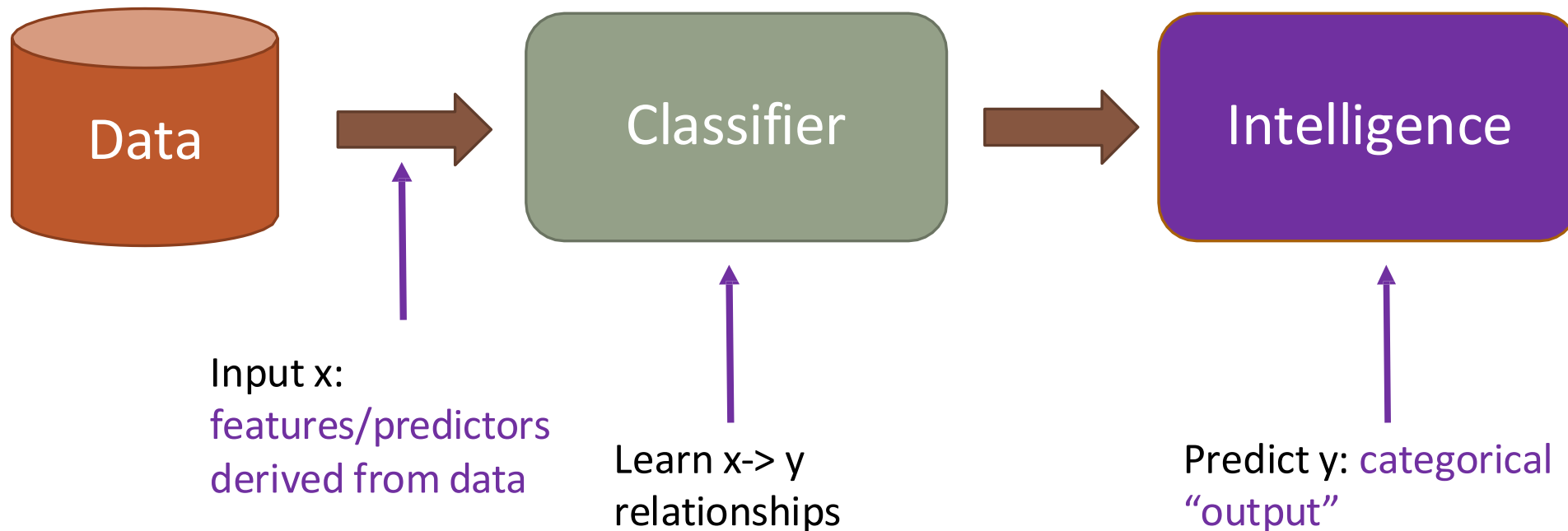


Lecture 3:Decision Tree

Classification: Recap

- Classify objects into a set of **pre-specified** classes (or categories) based on the values of relevant object attributes (features).



Overview

- Decision Tree Model
 - Step 1: Kicked Vehicle Data
 - Step 2: Data Preprocessing
 - Step 3: Build a decision tree on training data (**learn the relationship between predictors and target variable**)
 - Step 4: Evaluate Decision Tree Model Performance

Kicked Vehicle

- One of the biggest challenges of an auto dealership purchasing a used car at an auto auction is the risk of that the vehicle might have serious issues that prevent it from being sold to customers. The auto community calls these unfortunate purchases "kicks".
- Kicked cars often result when there are tampered odometers, mechanical issues the dealer is not able to address, issues with getting the vehicle title from the seller, or some other unforeseen problem. Kick cars can be very costly to dealers after transportation cost, throw-away repair work, and market losses in reselling the vehicle.

Kicked Vehicle

- Figure out which cars have a higher risk of being kick can provide real value to dealerships trying to provide the best inventory selection possible to their customers.
- Problem: Predict if the car purchased at the Auction is a Kick (bad buy).

What we know about a car?

- Auction: Auction provider at which the vehicle was purchased
- Color: Vehicle Color
- MMRCurrentAuctionAveragePrice: Acquisition price for this vehicle in average condition as of current day
- Size: The size category of the vehicle (Compact, SUV, etc.)
- TopThreeAmericanName: Identifies if the manufacturer is one of the top three American manufacturers
- VehBCost: Acquisition cost paid for the vehicle at time of purchase
- VehicleAge: The Years elapsed since the manufacturer's year
- VehOdo: The vehicles odometer reading
- WarrantyCost: Warranty price (term=36month and millage=36K)
- WheelType: The vehicle wheel type description (Alloy, Covers)

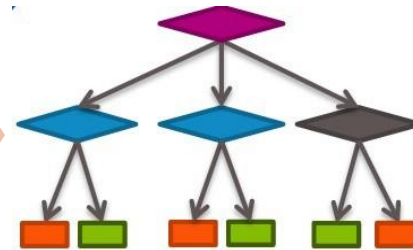
- What we don't know about a car?

What we know about a car?

Information of Cars

- Auction
- Color
- MMRCurrentAuctionAveragePrice
- Size
- TopThreeAmericanName
- VehBCost
- VehicleAge
- VehOdo
- WarrantyCost
- WheelType

Predictors (X)



Classifier
(decision tree model)

IsBadBuy

Target variable
/Label (Y)

Predictor and Target Variable

- **Predictor:** A predictor (or **predictor variable**) is a variable whose values will be used to predict the value of the target variable.
- **Target variable:** The target variable (or **outcome variable**) is the variable whose values are to be modeled and predicted by other variables.
- **Classifier:** an algorithm/classification model that implements classification (mapping input data to a category).

Steps to build a decision tree model

- Step 1: Kicked Vehicle Data
- Step 2: Data Preprocessing
- Step 3: Build a decision tree on training data (learn the relationship between predictors and target variable)
- Step 4: Evaluate Decision Tree Model Performance

Step 1: Kicked Vehicle Data

	Auction	Color	IsBadBuy	MMRCurrentAuctionAveragePrice	Size	TopThreeAmericanName	VehBCost	VehicleAge	VehOdo	WarrantyCost	WheelType
0	ADESA	WHITE	No	2871	LARGE TRUCK	FORD	5300	8	75419	869	Alloy
1	ADESA	GOLD	Yes	1840	VAN	FORD	3600	8	82944	2322	Alloy
2	ADESA	RED	No	8931	SMALL SUV	CHRYSLER	7500	4	57338	588	Alloy
3	ADESA	GOLD	No	8320	CROSSOVER	FORD	8500	5	55909	1169	Alloy
4	ADESA	GREY	No	11520	LARGE TRUCK	FORD	10100	5	86702	853	Alloy
...	...	...	...	...	...	...	...	...	...	...	...
9995	ADESA	RED	No	7536	SMALL SUV	CHRYSLER	6600	4	85377	983	Alloy
9996	ADESA	BLACK	No	4921	LARGE TRUCK	GM	7000	7	89665	1543	Alloy
9997	ADESA	BLACK	No	9263	MEDIUM SUV	CHRYSLER	9000	4	59383	1417	Alloy
9998	ADESA	BLUE	No	3240	MEDIUM	OTHER	5500	4	48642	482	Covers
9999	ADESA	SILVER	No	6148	MEDIUM SUV	FORD	7000	5	82563	1243	Alloy

10000 rows x 11 columns

Target variable

Steps to build a decision tree model

- Step 1: Kicked Vehicle Data
- **Step 2: Data Preprocessing**
- Step 3: Build a decision tree on training data (learn the relationship between predictors and target variable)
- Step 4: Evaluate Decision Tree Model Performance

Step 2: Data Preprocessing

- **Dummy coding (one-hot encoding)**

- To convert categorical variables into a numeric format
 - Create **dummy variables** to replace the original categorical variable
 - Dummy coding for a binary variable: gender

$$\text{male} = \begin{cases} 1 & \text{if } x = \text{male} \\ 0 & \text{otherwise} \end{cases}$$

Auction provider: ADESA, MANHEIM, OTHER

n categories, *n*-1 dummy variables

	Auction
Vehicle 1	OTHER
Vehicle 2	MANHEIM
Vehicle 3	ADESA



	Auction_MANHEIM	Auction_OTHER
Vehicle 1	0	1
Vehicle 2	1	0
Vehicle 3	0	0

Step 2: Data Preprocessing

Target variable

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	VehOdo	WarrantyCost	Auction_MANHEIM	Auction_OTHER	Color_BLACK	...	WheelType_unkwnWheel
0	No	2871	5300	8	75419	869	0	0	0		0
1	Yes	1840	3600	8	82944	2322	0	0	0		0
2	No	8931	7500	4	57338	588	0	0	0		0
3	No	8320	8500	5	55909	1169	0	0	0		0
4	No	11520	10100	5	86702	853	0	0	0		0
...	...	...	...	...	...	...	...	...	...	...	...
9995	No	7536	6600	4	85377	983	0	0	0		0
9996	No	4921	7000	7	89665	1543	0	0	1		0
9997	No	9263	9000	4	59383	1417	0	0	1		0
9998	No	3240	5500	4	48642	482	0	0	0		0
9999	No	6148	7000	5	82563	1243	0	0	0		0

10000 rows x 40 columns

Step 2: Data Preprocessing

Split the data to training and testing sets

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	VehOdo	WarrantyCost	Auction_MANHEIM	Auction_OTHER	Color_BLACK	...	WheelType_unkwnWheel	
0	No	2871	5300	8	75419	869	0	0	0		0	
1	Yes	1840	3600	8	82944	2322	0	0	0		0	
2	No	8931	7500	4	57338	588	0	0	0		0	
3	No	8320	8500	5	55909	1169	0	0	0		0	
4	No	11520	10100	5	86702	853	0	0	0		0	
...	...	...	...	...	...	...	...	...	...	...	...	
6995	No	2762	4185	4	84379	1243	0	1	0		0	
6996	Yes	5144	8705	5	80600	1373	0	1	0		0	
6997	No	2912	4485	6	77686	1389	0	1	0		0	
6998	No	4965	6495	4	66731	1020	0	1	0		0	
6999	No	2724	3985	6	86275	1243	0	1	0		0	
...	...	...	...	...	...	...	...	...	...	...	...	
9995	No	7536	6600	4	85377	983	0	0	0		0	
9996	No	4921	7000	7	89665	1543	0	0	1		0	
9997	No	9263	9000	4	59383	1417	0	0	1		0	
9998	No	3240	5500	4	48642	482	0	0	0		0	
9999	No	6148	7000	5	82563	1243	0	0	0		0	

Training data

Testing data

Step 2: Data Preprocessing

- **Imbalanced data and “fool’s gold”**
 - When one class **dominates** the dataset
 - Common in various domains (fraud detection, medical diagnosis, risk and safety)
 - For instance, we might have 99% non-fraud, 1% fraud in transactions
- Models can appear highly accurate (fool’s gold) while being useless
- Predicting only the majority class can yield high accuracy

Ways to address imbalanced data

- Resampling Techniques
- Balancing helps models learn meaningful patterns rather than majority-class bias
 - Undersampling: reduce the majority class
 - Oversampling: duplicate minority class
 - SMOTE: generate synthetic minority samples
- Algorithm-Level Approaches (Class weights, Cost-sensitive learning)
- Threshold Adjustment (Change the classification cutoff (not always 0.5))
- Use Appropriate Evaluation Metrics (Precision, Recall, F1-score, ROC-AUC)
-  Never balance the test set

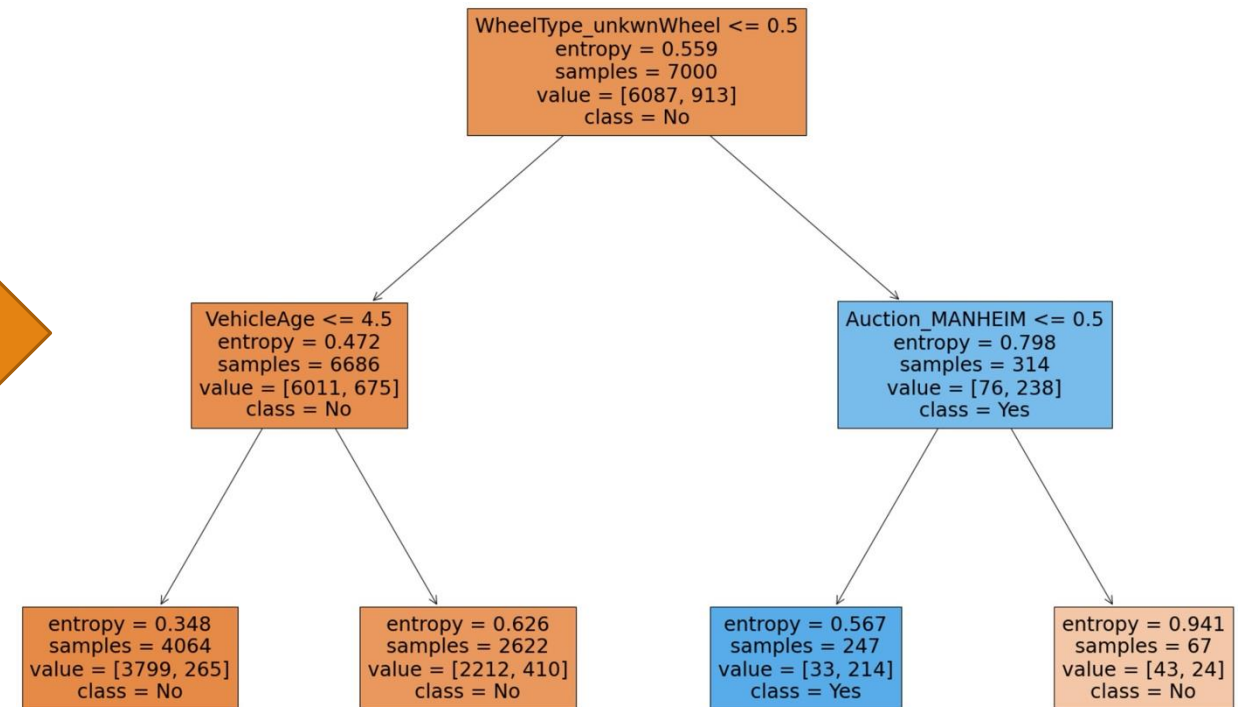
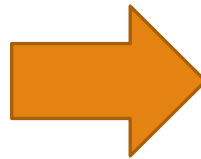
Steps to build a decision tree model

- Step 1: Kicked Vehicle Data
- Step 2: Data Preprocessing
- Step 3: Build a decision tree on training data (**learn the relationship between predictors and target variable**)
- Step 4: Evaluate Decision Tree Model Performance

Step 3: Build a decision tree on training data

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	...
0	No	2871	5300	8	
1	Yes	1840	3600	8	
2	No	8931	7500	4	
3	No	8320	8500	5	
4	No	11520	10100	5	
...	...	...	...	...	...
6995	No	2762	4185	4	
6996	Yes	5144	8705	5	
6997	No	2912	4485	6	
6998	No	4965	6495	4	
6999	No	2724	3985	6	

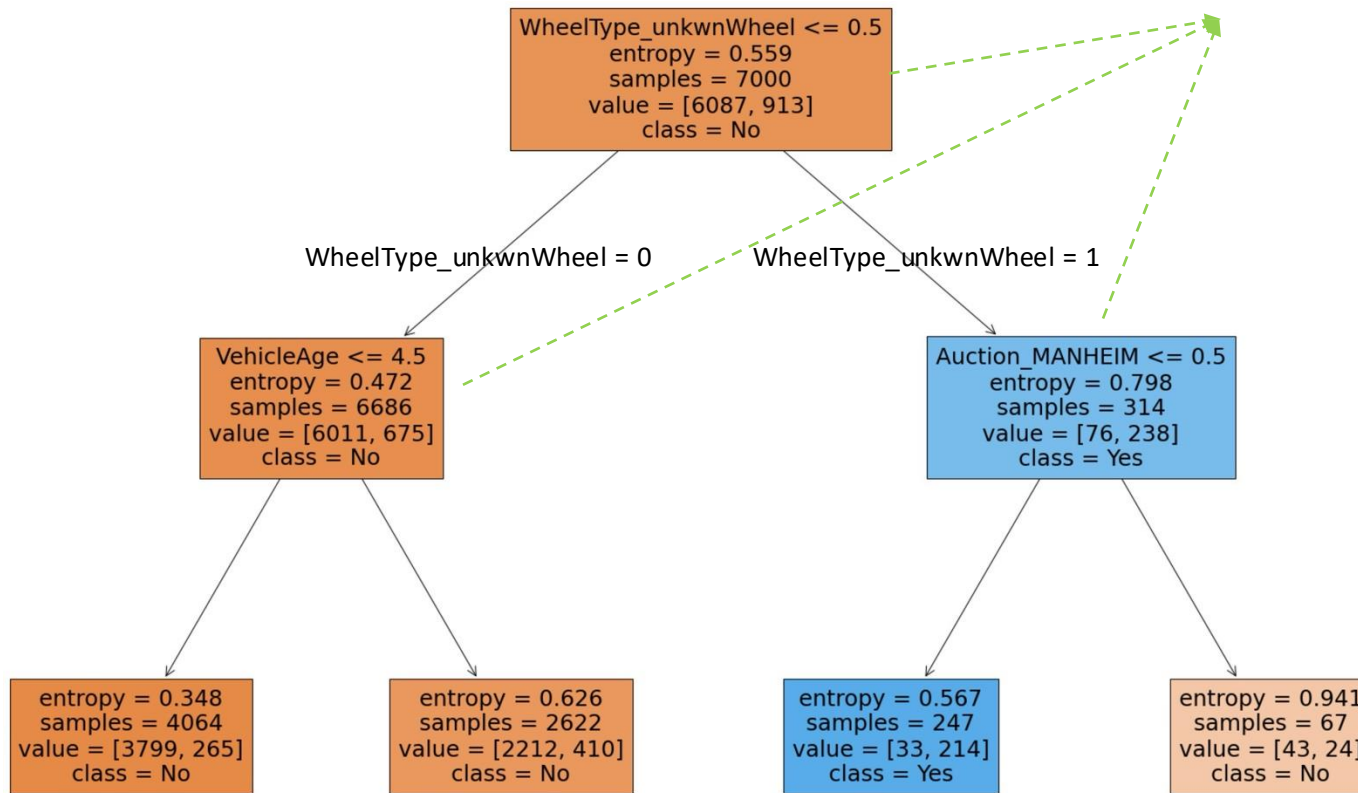
Training data



Decision Tree Model

Decision Tree

Decision nodes, including
root node and internal nodes



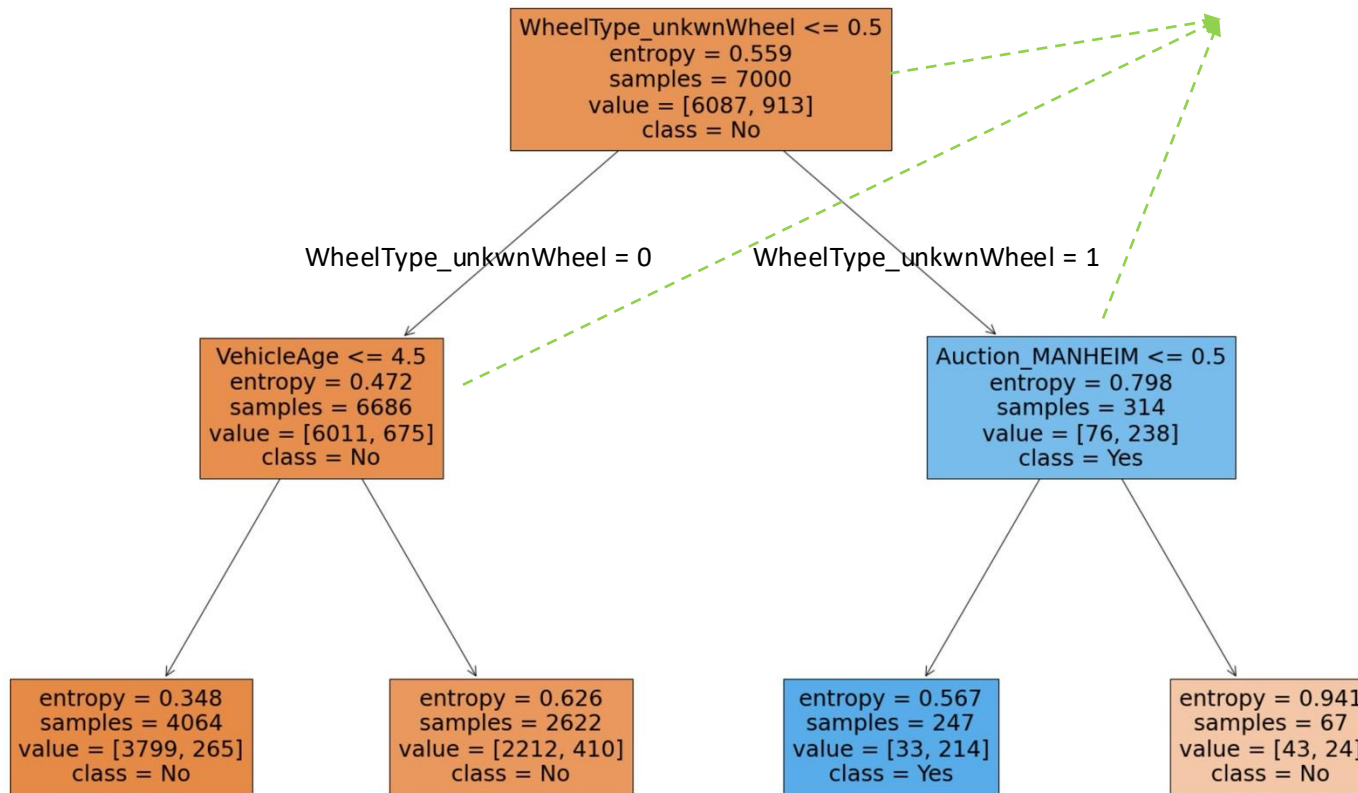
Leaf nodes

Decision Tree structure

- **Leaf Node:** The end of the tree. Nodes do not split is called Leaf or Terminal node.
 - A leaf node holds a class label (outcome): prediction result - Yes or No
- **Decision Node:** Includes both root node and internal node. The root node or an internal node contains a predictor.
 - Root Node: The first node of the tree. It represents entire population or sample and this further gets divided into two or more homogeneous sets.

Decision Tree

Decision nodes, including
root node and internal nodes



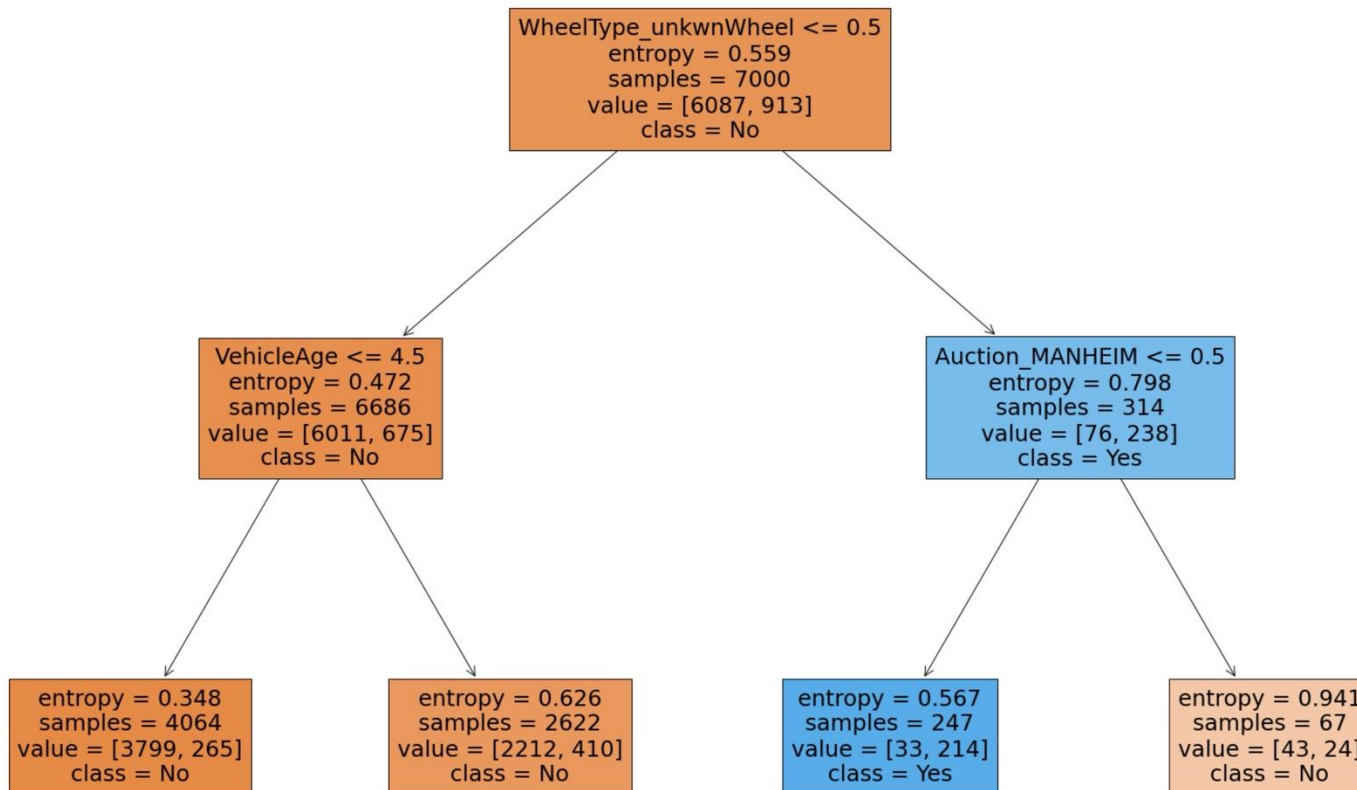
Leaf nodes

Classification rules

- A decision tree can be expressed as a set of IF-THEN rules.
- Each path from the root to a leaf forms an IF-THEN rule.
- Each observation/data record is classified based on an IF-THEN rule
 - IF WheelType = unkwnWheel, Auction = {ADESA, OTHER}, THEN IsBadBuy=YES

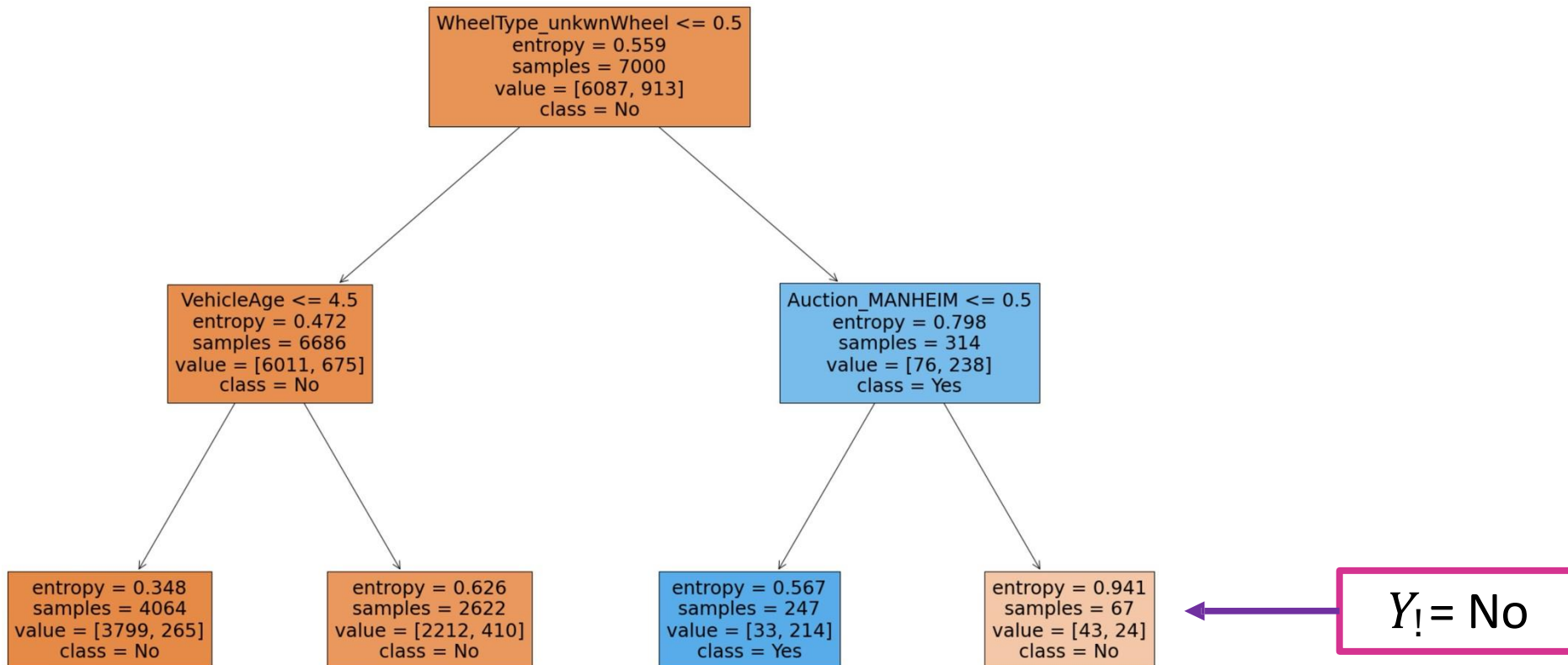
Decision Tree

$X_i = (\text{WheelType} = \text{unkwnWheel}, \text{Auction} = \text{MANHEIM})$



Decision Tree

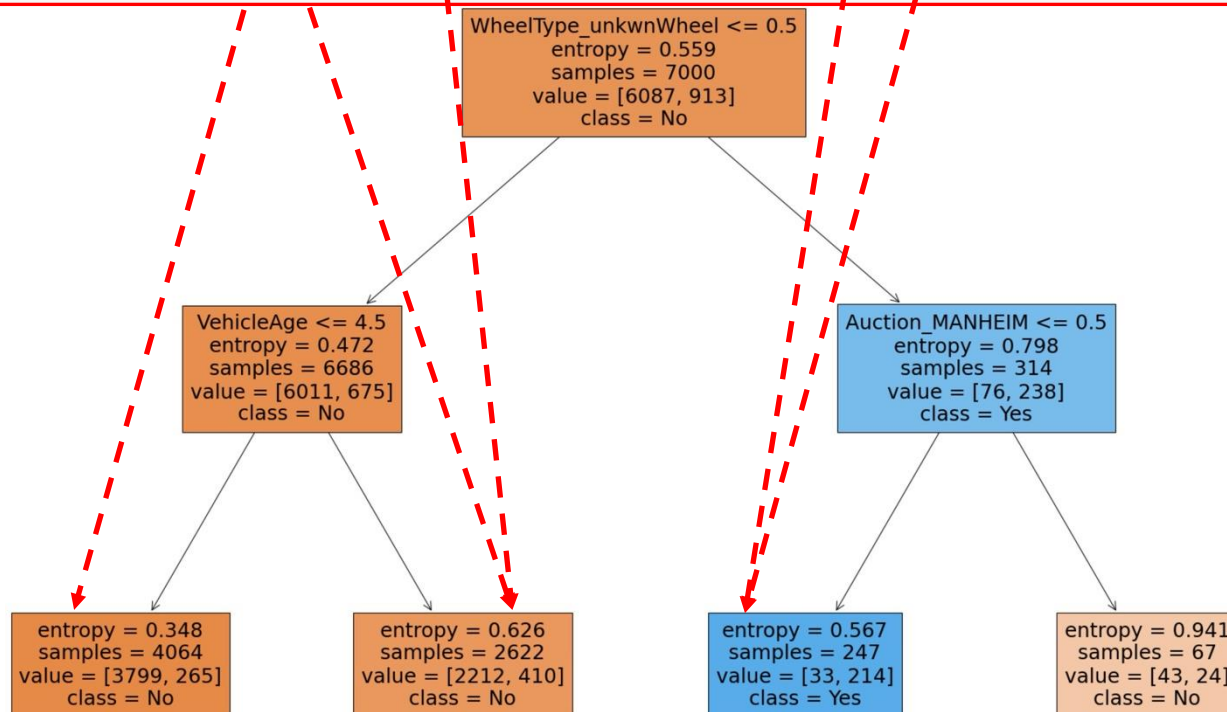
$X_i = (\text{WheelType} = \text{unkwnWheel}, \text{Auction} = \text{MANHEIM})$



Decision Tree

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	VehOdo	WarrantyCost	Auction_MANHEIM	Auction_OTHER	...	WheelType_Covers	WheelType_Special	WheelType_unkwnWheel
--	----------	-------------------------------	----------	------------	--------	--------------	-----------------	---------------	-----	------------------	-------------------	----------------------

0		2871	5300	8	75419	869	0	0		0	0	0
1		1840	3600	8	82944	2322	0	0		0	0	1
2		8931	7500	4	57338	588	0	0		0	0	0
3		8320	8500	5	55909	1169	0	0	...	0	0	1
4		11520	10100	5	86702	853	0	0		0	0	0



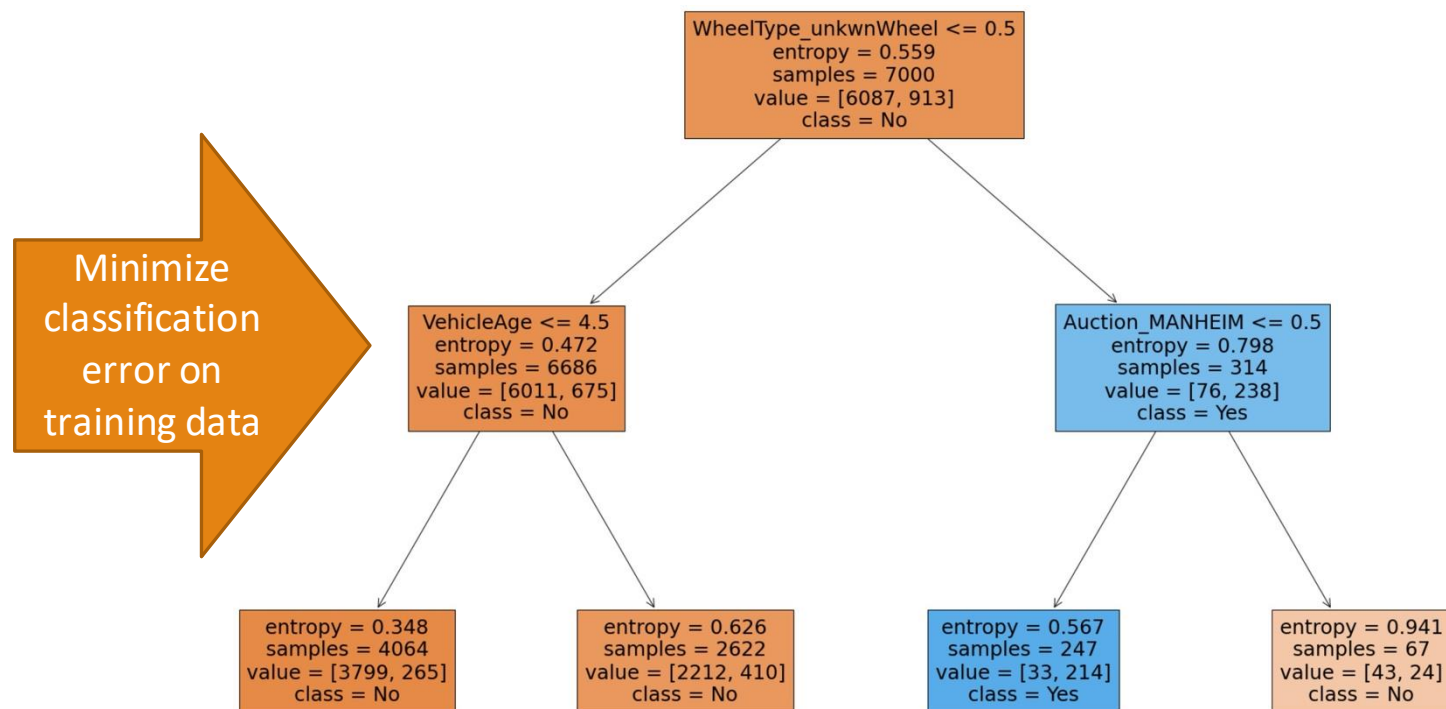
Decision tree splits the dataset

Build a Decision Tree (learn the relationship between predictors and target variable)

- Training data: 7,000 observations (X_i, Y_i)

	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	...
0	No	2871	5300	8	
1	Yes	1840	3600	8	
2	No	8931	7500	4	
3	No	8320	8500	5	
4	No	11520	10100	5	
...	...	...	...	...	...
6995	No	2762	4185	4	
6996	Yes	5144	8705	5	
6997	No	2912	4485	6	
6998	No	4965	6495	4	
6999	No	2724	3985	6	

Training data



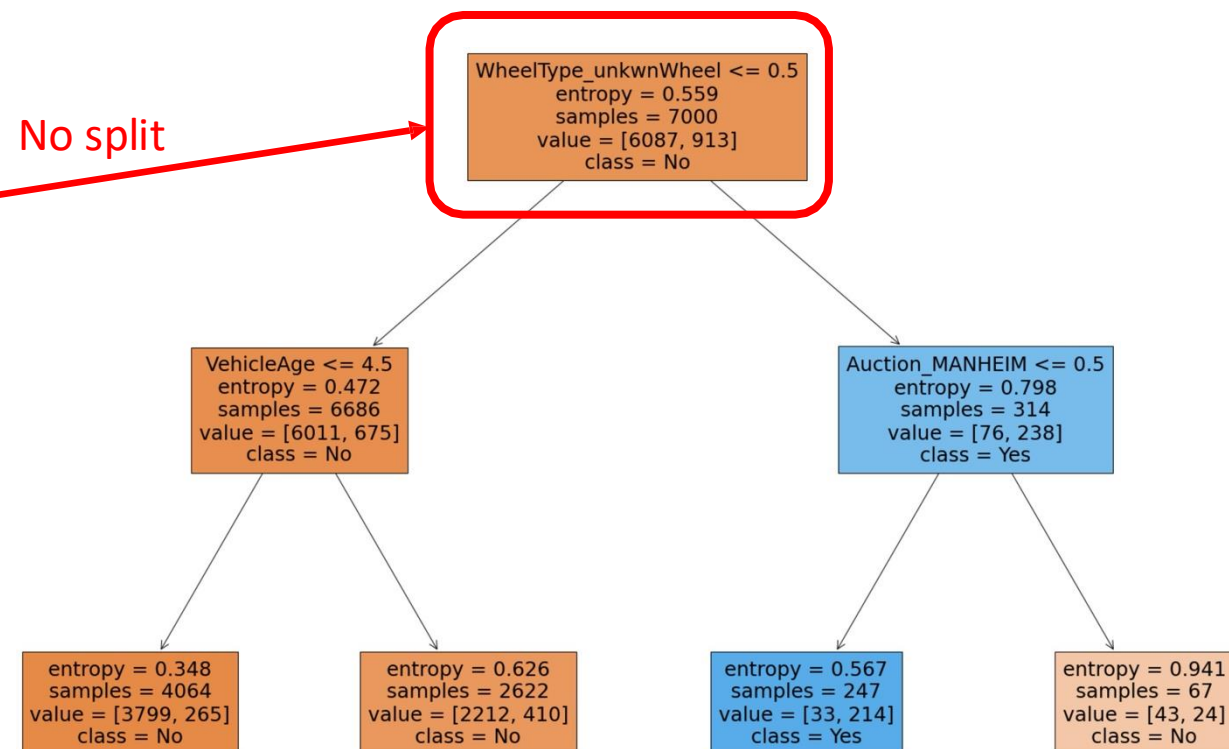
Decision Tree Model

Build a Decision Tree (learn the relationship between predictors and target variable)

- Key step : Select a predictor to split data

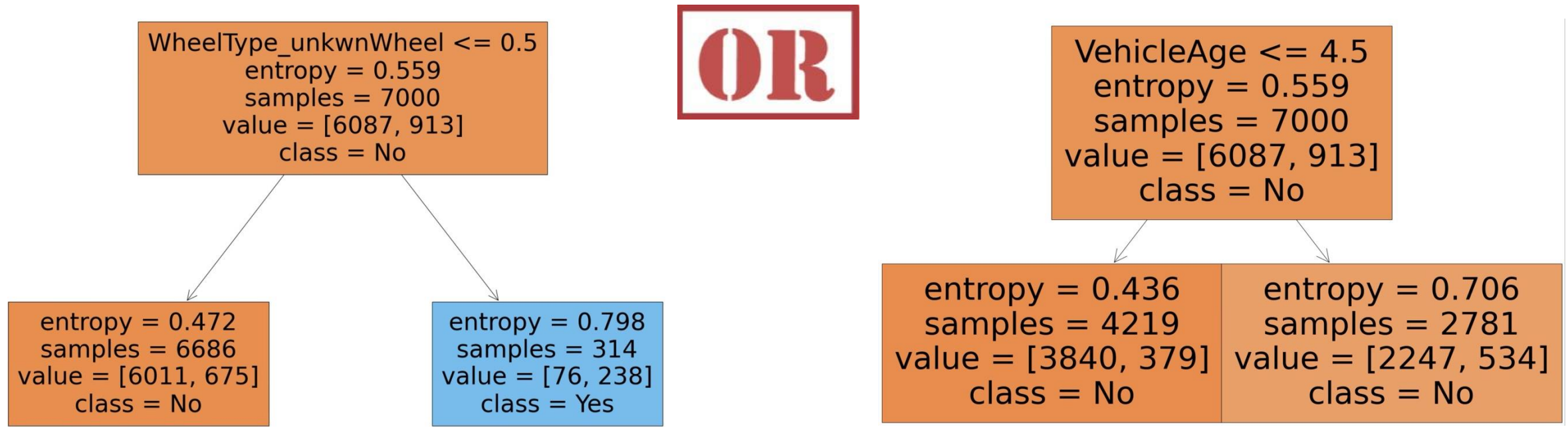
	IsBadBuy	MMRCurrentAuctionAveragePrice	VehBCost	VehicleAge	...
0	No	2871	5300	8	
1	Yes	1840	3600	8	
2	No	8931	7500	4	
3	No	8320	8500	5	
4	No	11520	10100	5	
...	...	...	...	...	...
6995	No	2762	4185	4	
6996	Yes	5144	8705	5	
6997	No	2912	4485	6	
6998	No	4965	6495	4	
6999	No	2724	3985	6	

Training data



Build a Decision Tree (learn the relationship between predictors and target variable)

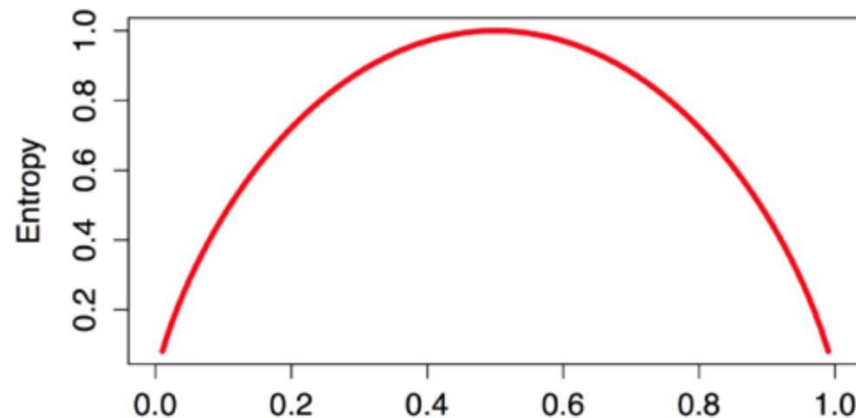
- Key step : Select a predictor to split data



Entropy

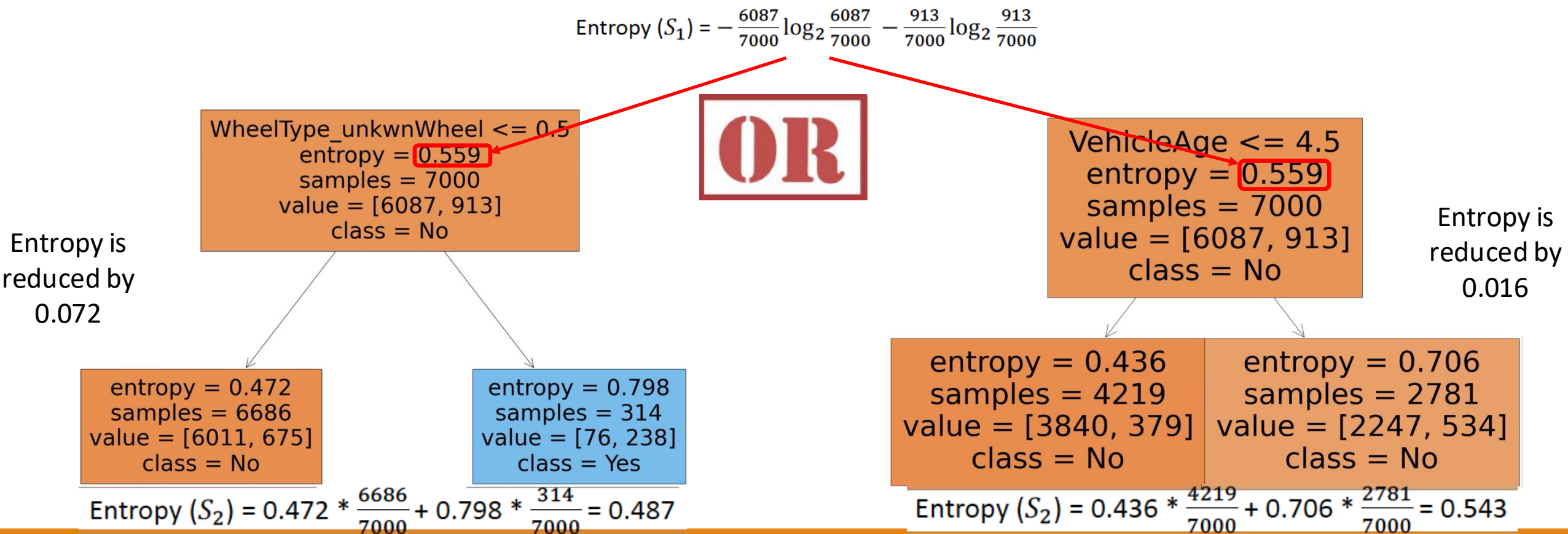
- The degree to which a subset of examples contains only a single class is known as **purity**
- Measure the purity
 - **Entropy**, a concept borrowed from information theory that quantifies the randomness, or disorder, within a set of class values.

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$



Build a Decision Tree (learn the relationship between predictors and target variable)

- Key step : Select a predictor to split data



Build a Decision Tree (learn the relationship between predictors and target variable)

- To use entropy to determine the optimal feature to split upon, the algorithm calculates the change in homogeneity, which is a measure known as **information gain**. The information gain for a feature F is calculated as the difference between the entropy in the segment before the split (S_1) and the partitions resulting from the split (S_2):

$$\text{InfoGain}(F) = \text{Entropy}(S_1) - \text{Entropy}(S_2)$$

$$\text{Entropy}(S) = \sum_{i=1}^n w_i \text{Entropy}(P_i)$$

Build a Decision Tree (learn the relationship between predictors and target variable)

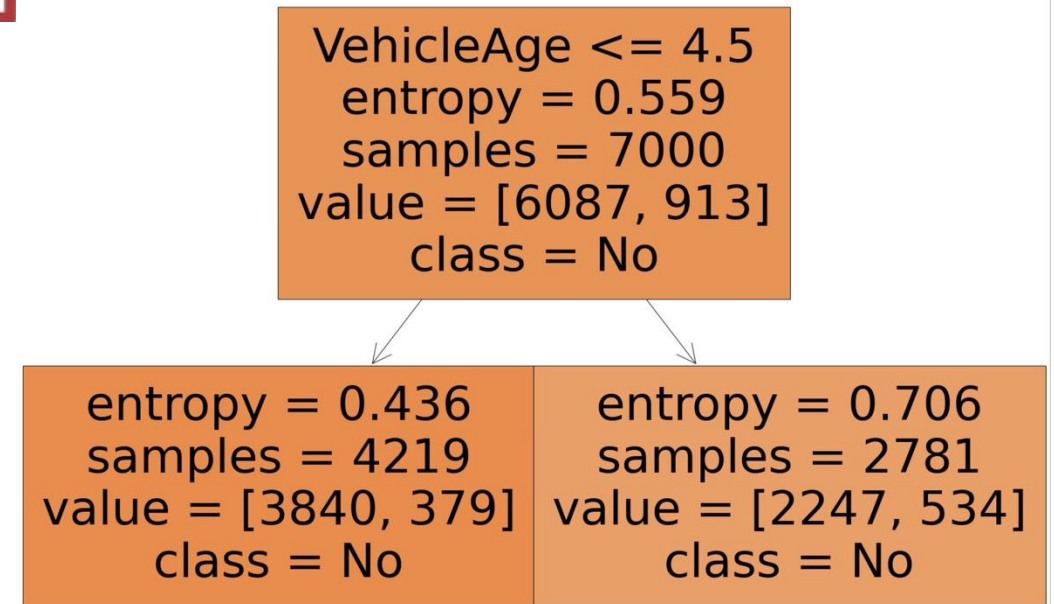
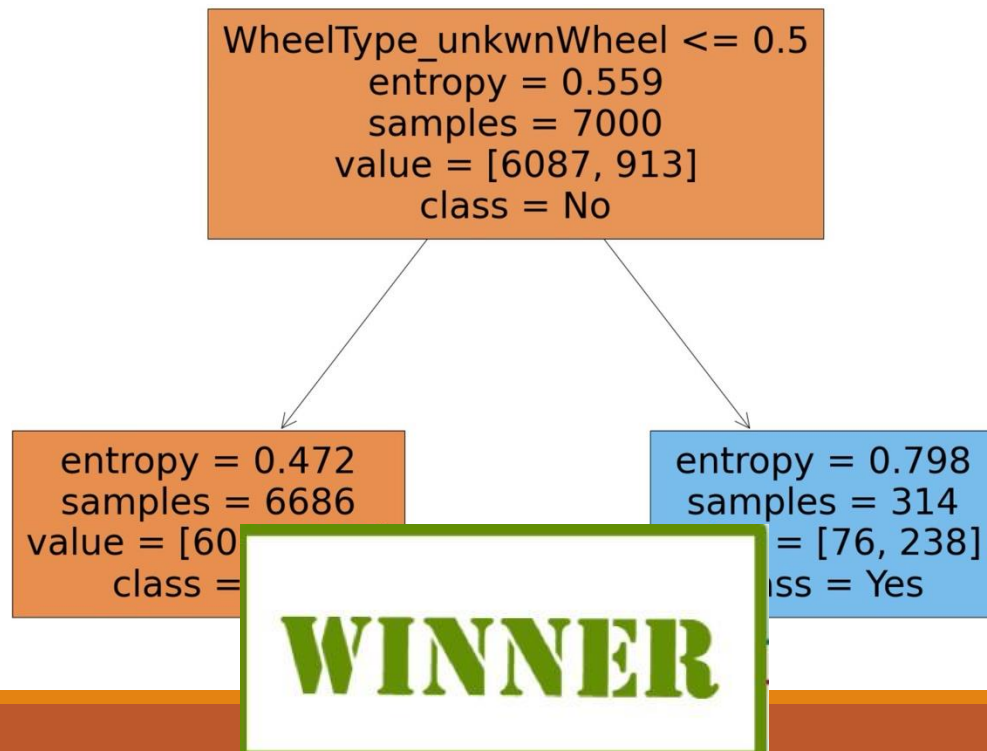
- The higher the information gain, the better a feature is at creating homogeneous groups after a split on this feature. If the information gain is zero, there is no reduction in entropy for splitting on this feature.

Higher information gain = lower entropy

Build a Decision Tree (learn the relationship between predictors and target variable)

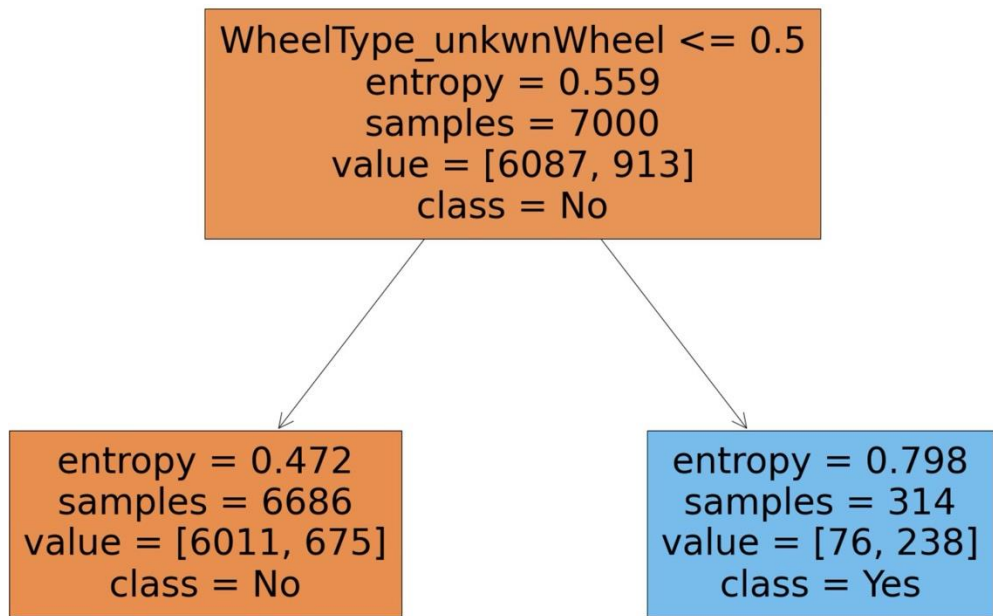
- Key step : Select a predictor to split data

OR

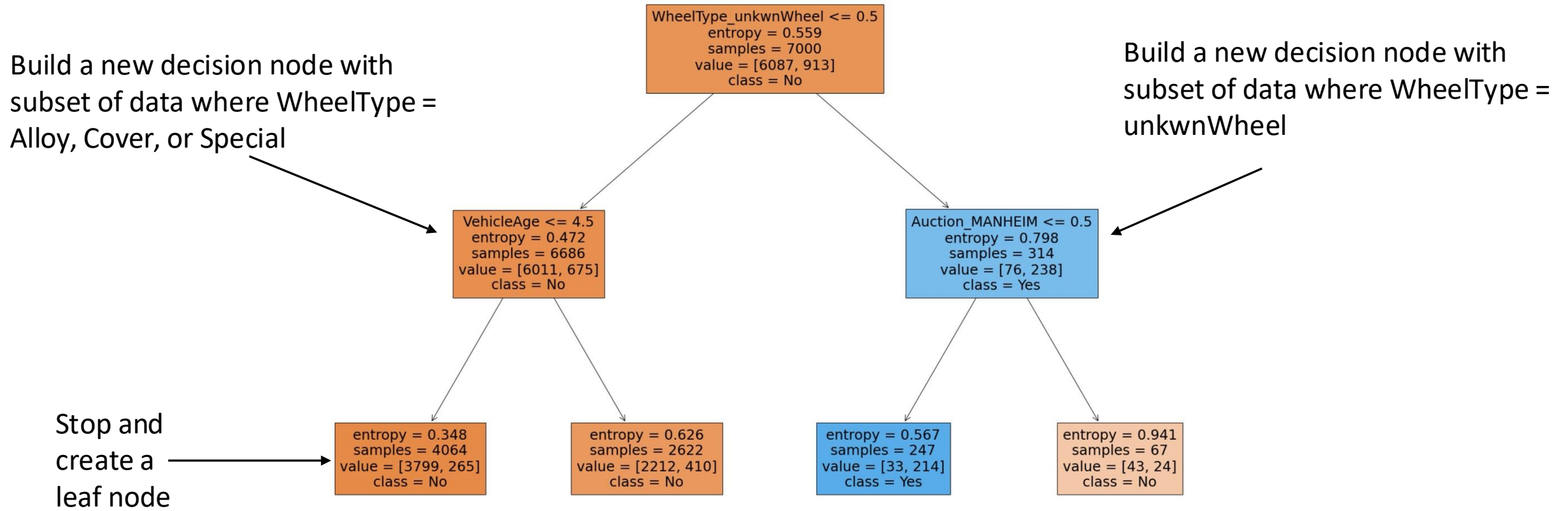


Build a Decision Tree (learn the relationship between predictors and target variable)

- Continue to split
 - Build decision node with subset of data where WheelType = Alloy, Cover, or Special (left path)
 - Build decision node with subset of data where WheelType = unkwnWheel (right path)



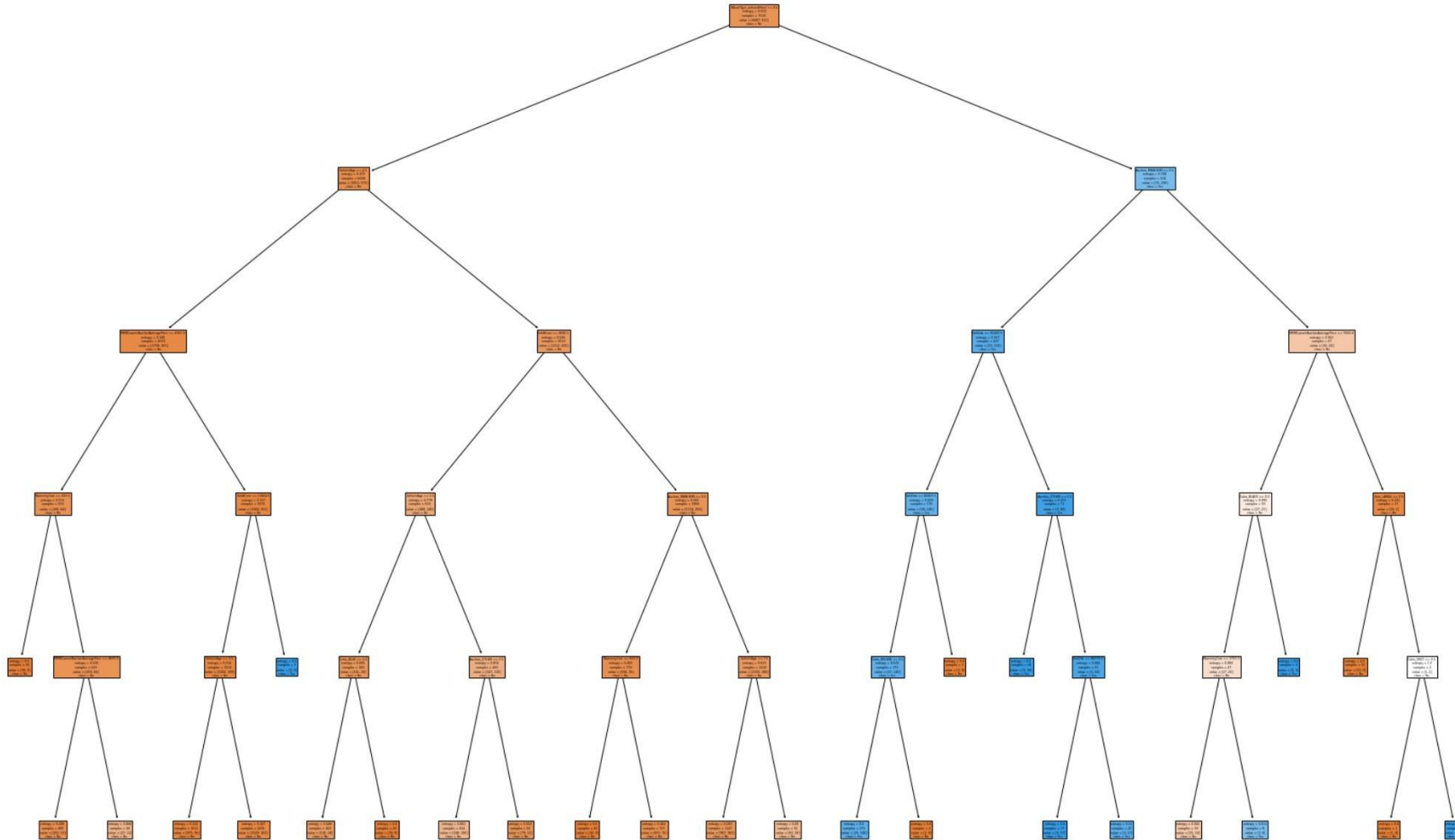
Build a Decision Tree (learn the relationship between predictors and target variable)



Build a Decision Tree (learn the relationship between predictors and target variable)

- A decision tree can continue to grow indefinitely, choosing splitting predictors and dividing the data into smaller and smaller partitions.
 - However, if the tree grows overly large, many of the decisions it makes will be overly specific and the model will be overfitted to the training data.

Build a Decision Tree (learn the relationship between predictors and target variable)



Build a Decision Tree (learn the relationship between predictors and target variable)

- Greedy Approach to find a “good” tree
 - Step 1: Start with an empty tree
 - Step 2: Select a predictor with highest information gain to split data
 - Step 3: Create a branch for each value of the split attribute and according to this, divide the data set into several subsets.
 - Step 4: For each subset:
 - If nothing more to do, create a leaf node
 - Otherwise, go to Step 2 & continue (recurse) to split subset

Recursion



Other splitting criteria

- **Gini Index (Gini Impurity)**

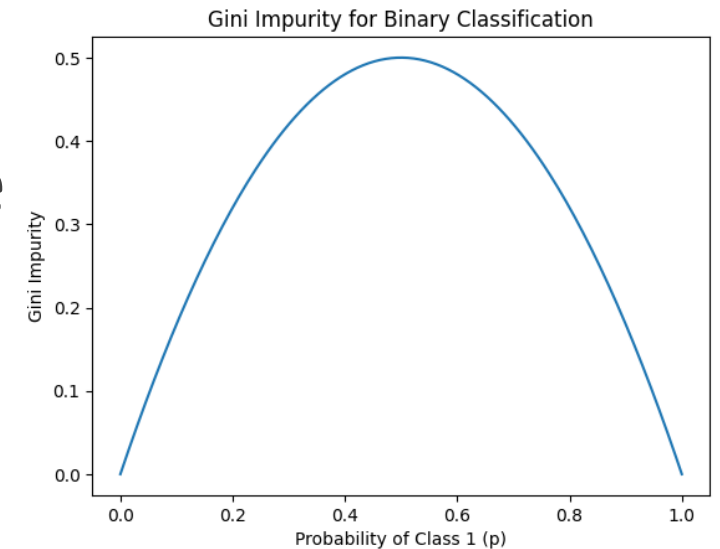
- Measures how mixed the classes are in a node
- Lower Gini = purer node

Formula:

$$\text{Gini} = 1 - \sum (p_i)^2$$

where p_i is the proportion of class i in the node

- Commonly used in **CART** (Classification and Regression Trees) and **Random Forests**



Other splitting criteria

- **Log Loss (Cross-Entropy Loss)**

- Measures how well predicted probabilities match actual classes
- Penalizes confident wrong predictions

Binary classification formula:

$$\text{Log Loss} = - [y \cdot \log(p) + (1 - y) \cdot \log(1 - p)]$$

- Used when trees output **probabilities** (e.g., boosting)

Other splitting criteria

- Information Gain Ratio
- Misclassification Error
- Chi-Square Statistic
- Variance Reduction (Regression Trees)
- Mean Squared Error (MSE)
- ANOVA / F-test

Training Stopping / Termination Conditions

❑ Stopping conditions prevent **overfitting** and control tree complexity

- ❑ **Pure Node:** All samples in the node belong to the same class (Entropy = 0)
- ❑ **Maximum Tree Depth Reached**
- ❑ **Minimum Samples per Node**
- ❑ **Minimum Samples per Split**
- ❑ **No Significant Impurity Reduction**
All Features Exhausted

❑ Pre-pruning vs Post-pruning

- ❑ **Pre-pruning:** stop early using rules above
- ❑ **Post-pruning:** grow full tree, then prune back

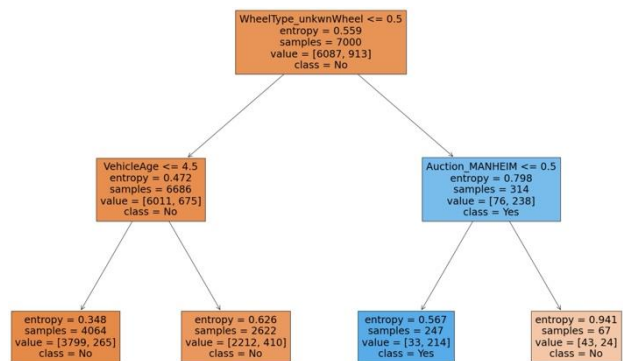
Steps to build a decision tree model

- Step 1: Kicked Vehicle Data
- Step 2: Data Preprocessing
- Step 3: Build a decision tree on training data (learn the relationship between predictors and target variable)
- Step 4: Evaluate Decision Tree Model Performance

Step 4: Evaluate Decision Tree Model Performance

- Holdout Evaluation
 - Splitting method (percentage split)- divide into training and testing sets (e.g. 70%/30% or 2/3 to 1/3)
 - 2 partitions, e.g.
 - 70% and 30% in training and testing
 - **Training and testing data are not overlapped**
 - Most commonly used to get a sense of a model's performance level

Step 4: Evaluate Decision Tree Model Performance



...	...	...	...	...	...	...	...	...	...
9995		7536	6600	4	85377	983	0	0	0
9996		4921	7000	7	89665	1543	0	0	1
9997		9263	9000	4	59383	1417	0	0	1
9998		3240	5500	4	48642	482	0	0	0
9999		6148	7000	5	82563	1243	0	0	0

Testing data

Predictions	Real value
IsBadBuy	IsBadBuy
No	No
No	No
No	No
No	No
Yes	No
Yes	No

Step 4: Evaluate Model Performance

- Compare the **predictions** and **real values/actual value**

Predictions/predicted values

IsBadBuy
No
No
No
No
Yes
Yes

real values

IsBadBuy
No
No
No
No
No
No

Step 4: Evaluate Model Performance

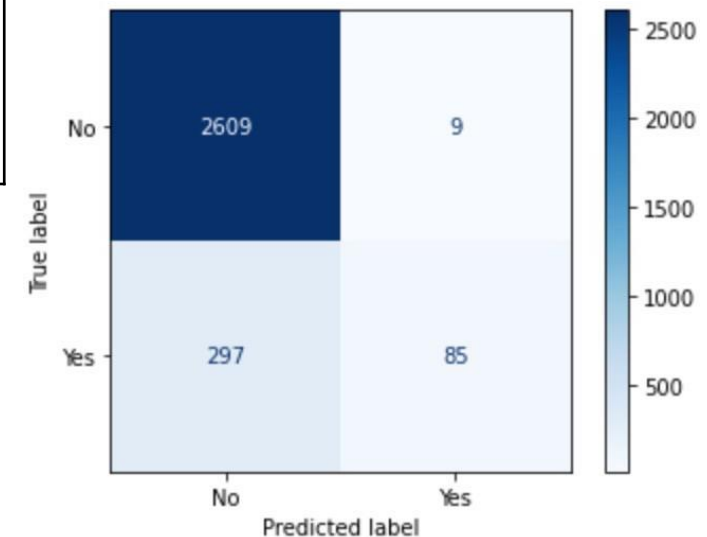
- Confusion Matrix
 - A **confusion matrix** is a table that categorizes predictions according to whether they match the actual value.

Step 4: Evaluate Model Performance

	Predicted Class Label		
		a	b
	True Class Label	a	b
	a	True Positive (TP)	False Negative (FN) (Type II error)
	b	False Positive (FP) (Type I error)	True Negative (TN)

- **True Positive (TP)**: Correctly classified as is positive class
- **True Negative (TN)**: Correctly classified as negative class
- **False Positive (FP)**: Incorrectly classified as positive class
- **False Negative (FN)**: Incorrectly classified as negative class

Assume **a** is positive class and **b** is negative class



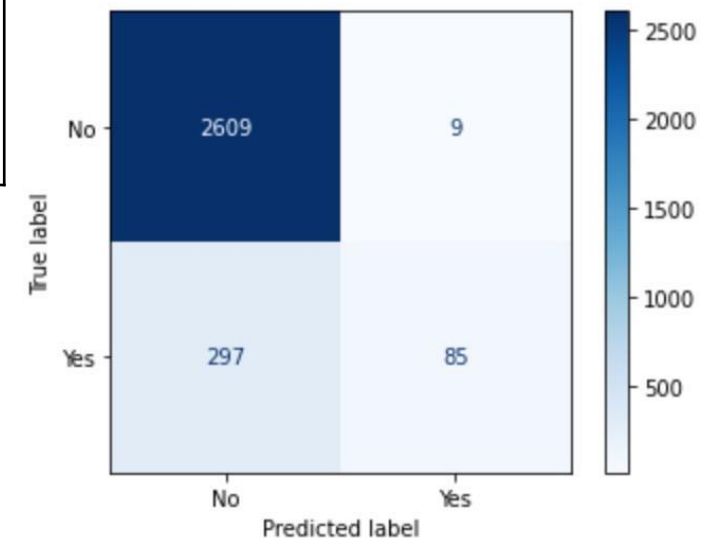
Step 4: Evaluate Model Performance

	Predicted Class Label		
		a	b
	True Class Label	a	b
	a	True Positive (TP)	False Negative (FN) (Type II error)
	b	False Positive (FP) (Type I error)	True Negative (TN)

- **a** is positive class
- **b** is negative class
- T (Total population) = $TP + TN + FP + FN$

- True class label is **a** = $TP + FN$
- Predicted class label is **a** = $TP + FP$
- True class label is **b** = $FP + TN$
- Predicted class label is **b** = $FN + TN$

Assume **a** is positive class and **b** is negative class



Step 4: Evaluate Model Performance

- **Accuracy** is the overall correctness of the model and is calculated as the sum of correct predictions divided by the total number of predictions.

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- The **error rate** or the proportion of the incorrectly classified examples is specified as

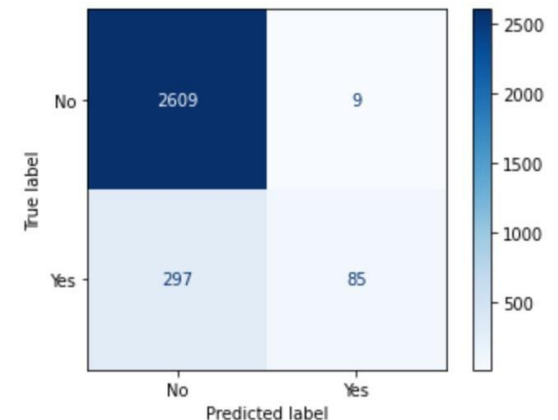
$$\text{error rate} = \frac{\text{FP} + \text{FN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = 1 - \text{accuracy}$$

Step 4: Evaluate Model Performance

- Evaluate performances on positive/negative class
 - **Precision**
 - **recall**
 - **F-measure**

Step 4: Evaluate Model Performance

- Evaluation Metrics: Precision
 - The **precision** is defined as the proportion of predicted positive examples that are truly positive.
 - How many instances are predicted to the positive (**a**) class?
 - Predicted class label is **a** = TP+FP
 - How many of instances predicted as **a** actually belong to **a** class?
 - TP
 - Precision (**a** or positive) = $TP / (TP + FP) = 2609 / (2609 + 297)$
 - Precision (**b** or negative) = $TN / (TN + FN) = 85 / (85 + 9)$



Step 4: Evaluate Model Performance

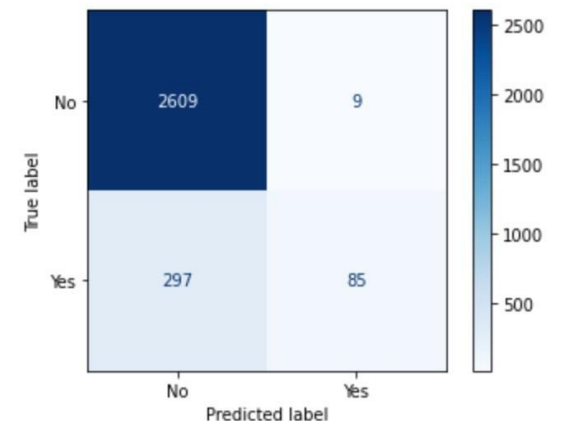
- Evaluation Metrics: Recall

- **Recall** is the ability to correctly classify instances belonging to this class.

- How many instances actually belong to the positive (**a**) class?
 - True class label is **a** = TP+FN
- How many of instances predicted as **a** actually belong to **a** class?
 - TP

- Recall (a or positive) = $TP / (TP + FN) = 2609 / (2609 + 9)$

- Recall (b or negative) = $TN / (TN + FP) = 85 / (85 + 297)$



Step 4: Evaluate Model Performance

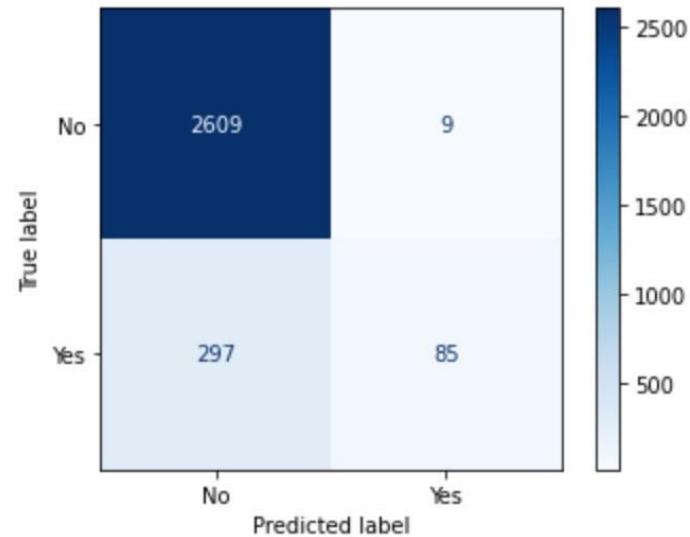
- F-measure

$$F_{\beta} = (1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

- The harmonic mean of precision and recall.
- It can be used as a single measure of overall performance on positive/negative class.
 - $F_1 \text{ Score } (\mathbf{a}) = (2 \times \text{Precision}(\mathbf{a}) \times \text{Recall}(\mathbf{a})) / (\text{Precision}(\mathbf{a}) + \text{Recall}(\mathbf{a}))$
 - $F_1 \text{ Score } (\mathbf{b}) = (2 \times \text{Precision}(\mathbf{b}) \times \text{Recall}(\mathbf{b})) / (\text{Precision}(\mathbf{b}) + \text{Recall}(\mathbf{b}))$
 - Where can an F_2 Score be useful?

Step 4: Evaluate Model Performance

Performance on the **testing data**:



	precision	recall	f1-score	support	
No	0.90	1.00	0.94	2618	Overall performance on Not bad buy class
Yes	0.90	0.22	0.36	382	Overall performance on bad buy class
accuracy			0.90	3000	
macro avg	0.90	0.61	0.65	3000	
weighted avg	0.90	0.90	0.87	3000	Model's overall performance

Step 4: Evaluate Model Performance

ROC(Receiver Operating Characteristic) Curve & AUC (Area Under the Curve)

- Plots **True Positive Rate (Recall)** vs **False Positive Rate**
- Shows model performance across **all classification thresholds**
- ROC shows the trade-off between detecting positives and making false alarms across thresholds.

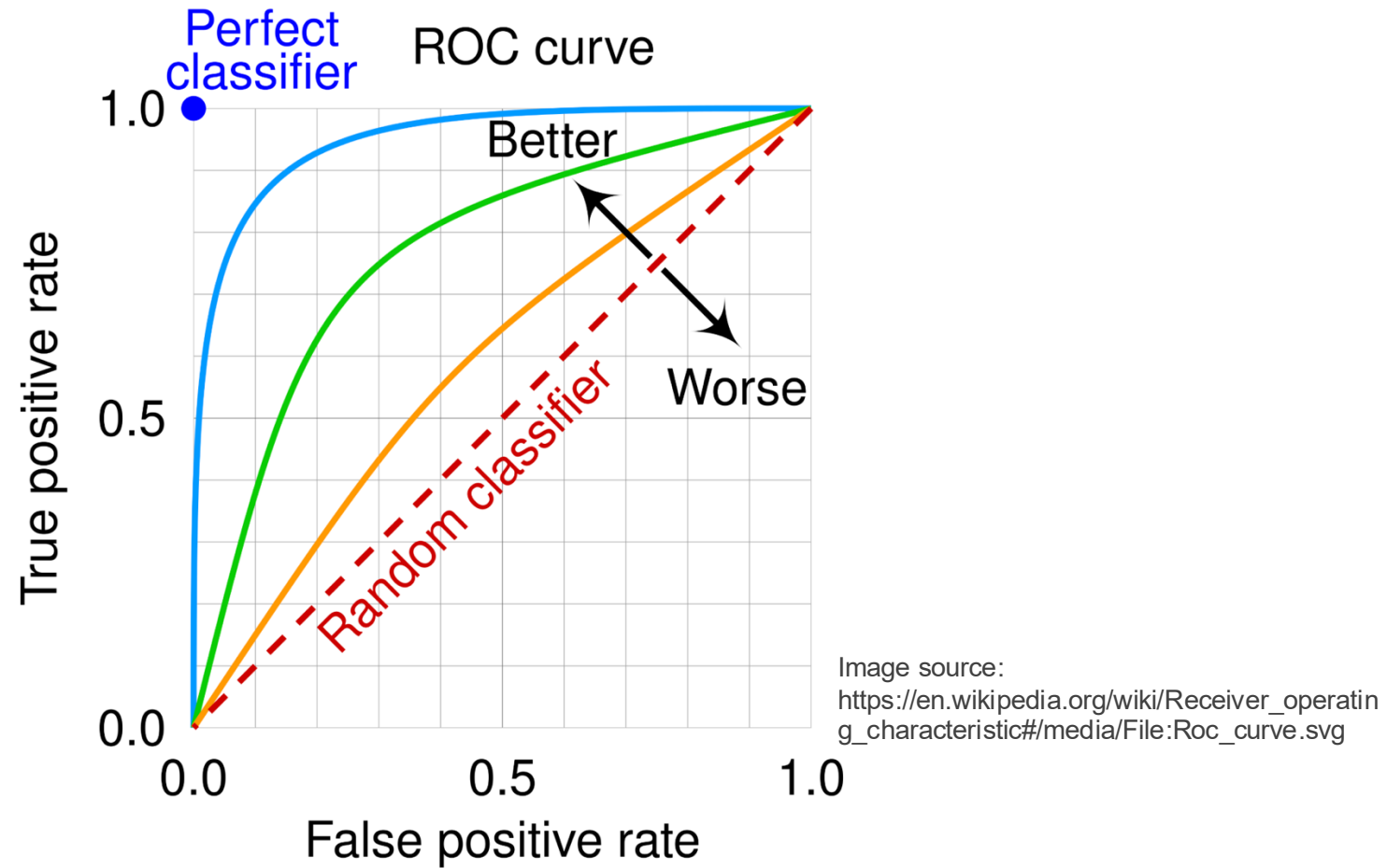
True Positive Rate (TPR) = $TP / (TP + FN)$

False Positive Rate (FPR) = $FP / (FP + TN)$

- How can we have ROC when there are more than two classes?

Step 4: Evaluate Model Performance

ROC Curve



- How can we have ROC when there are more than two classes?